A PRACTICAL LEARNING MAP FOR YOUR ACTUAL CODEBASE

Learn the Stack Behind Your Portfolio

A beginner-friendly guide to the languages, libraries, tools, and deployment workflow used by your biomedical engineering portfolio.

This site is a static React application. You can learn it one layer at a time: web foundations, TypeScript, React, Vite, styling, then GitHub deployment.

> **Your stack at a glance**
> HTML + CSS + JavaScript/TypeScript + React + Vite + Lucide React + GitHub Actions + GitHub Pages.

The guide separates what the site uses now from optional dependencies that are installed but not currently used.

Prepared for Eng Doughlas | Portfolio repository: dag-gieh/portfolio | Links are clickable in the PDF.

1. What the pieces do

Think of a framework as one part of a larger toolchain. React builds the interface; TypeScript checks your code; Vite runs the local development server and creates the production build; GitHub Actions publishes that build.

index.html
Browser page shell and app mount point.

src/main.tsx
Starts React and imports the app and CSS.

src/App.tsx
Contains portfolio data, components, state, events, and page sections.

src/index.css
Controls layout, colors, responsive behavior, and animation.

Vite
Serves the app locally, transforms modules, and builds the dist folder.

GitHub Actions + Pages
Builds and publishes dist when code is pushed to main.

Languages and formats

Technology	Where it appears	What to learn first
HTML	index.html; JSX inside .tsx files	Elements, attributes, forms, semantic structure
CSS	src/index.css	Selectors, box model, flexbox, grid, media queries
JavaScript	Logic inside src/App.tsx	Variables, objects, arrays, functions, modules, map/filter
TypeScript	.ts and .tsx files, tsconfig.json	Types, object shapes, unions, function and event types
JSX	React markup inside App.tsx	Components, props, expressions, lists, event handlers
YAML	.github/workflows/deploy.yml	Indentation, triggers, jobs, steps, permissions

> **Not in this project:** there is no server-side backend, database, Next.js, or React Router. The published site is static. Node.js is used to run development/build tools, not as a web server for visitors.

2. Learn the codebase in order

1. **Web foundations.** Learn semantic HTML, CSS layout, browser devtools, and JavaScript basics. Practice by changing a heading, a color token, and a responsive layout rule.
2. **TypeScript.** Read the types at the top of App.tsx: ProcessStep, Project, and the skill group type. Practice changing one data object and notice how type errors guide you. tsconfig.json sets strict checking; vite-env.d.ts supplies Vite's browser types.
3. **React and JSX.** Learn function components, props, rendering arrays with map, conditional rendering, events, and Hooks. In this site, useState drives theme, filters, map selection, and menus; useEffect connects to browser APIs.
4. **Vite and npm.** Learn package.json scripts, the dev server, the production build, and the public asset base path. vite.config.ts registers the React plugin. index.html is Vite's entry document.
5. **CSS and accessibility.** Study CSS variables, grid/flexbox, breakpoints, focus states, labels, and reduced-motion support. The site uses CSS animations and IntersectionObserver; it does not currently import Motion.
6. **Dependencies.** lucide-react provides typed icon components. The motion package is listed as a dependency but is not currently used in App.tsx. Learn it later only if you want more complex React-driven animation.
7. **Git and deployment.** Learn status, diff, add, commit, and push. The workflow deploy.yml runs on pushes to main: it installs dependencies, builds dist, uploads it, then publishes through GitHub Pages.

Where to make common edits

Goal	Start here
Change profile copy or statistics	portfolio object near the top of src/App.tsx
Change skills, projects, jobs, or education	Arrays in src/App.tsx
Change colors, spacing, mobile layout	Variables and rules in src/index.css
Change browser title/description	index.html
Change packages or scripts	package.json, then update the lockfile with npm
Change what happens on each push	.github/workflows/deploy.yml

3. A hands-on study plan

Use 30-45 minutes per session. Read a short lesson, then make one small change in your local copy and check the result.

Week 1: HTML, CSS, JavaScript

Inspect index.html. Change one CSS variable. Use browser devtools to inspect a card. Practice arrays and objects using the portfolio data.

Week 2: TypeScript + React

Trace main.tsx into App. Create a tiny component, pass it a typed prop, render a list, then add a small useState toggle.

Week 3: App interactions

Trace the project filter from button click to filtered cards. Trace the theme toggle and the mobile menu. Add clear labels and test keyboard focus.

Week 4: Build and publish

Run the production build. Read deploy.yml line by line. Make a small copy edit, review the diff, commit, push, then inspect the Actions run.

Small practice exercises

1. Change a skill label and confirm the page updates.
2. Add a project using the existing Project type; include only verified facts.
3. Add a category filter and ensure the project count changes correctly.
4. Make one mobile CSS adjustment and test at a narrow viewport.
5. Use git diff before committing, then verify the Pages deployment after pushing.

Windows PowerShell commands used in this repo

```
npm.cmd install
npm.cmd run dev
npm.cmd run build
git status
git diff
git add src/App.tsx src/index.css
git commit -m "Describe the change"
git push
```

Use npm.cmd in PowerShell if the npm.ps1 script is blocked by Windows execution policy. The dev server prints a localhost URL. A successful push to main starts deployment automatically.

4. Curated learning links

Start with the official documentation. Use videos when you want a narrated walkthrough, then return to the repo and practice.

HTML: MDN structuring content

Semantic elements, headings, links, forms, and document structure.

CSS: MDN styling basics

Selectors, cascade, box model, and the first styling concepts.

JavaScript: MDN Guide

Language fundamentals, functions, objects, arrays, and modules.

React: official Quick Start and Learn path

Components, JSX, data, lists, events, state, and sharing state.

React + TypeScript: official guide

Typed component props, hooks, event types, and React-specific patterns.

TypeScript: official Handbook

Begin with everyday types and object types; use the TSConfig reference when reading compiler settings.

Vite: official Getting Started

Dev server, index.html, npm scripts, config, production builds, and assets.

Lucide React: official guide

Import icons as React components and learn sizing and accessibility.

Motion for React: official docs

Optional for this repo; Motion is installed but its components are not currently used.

5. Videos, Git, and publishing

YouTube learning searches

Beginner web foundations: freeCodeCamp HTML/CSS/JavaScript

Choose a recent beginner course; practice the HTML and CSS ideas in the portfolio.

React + TypeScript: freeCodeCamp course search

Look for a current React + TypeScript build-along and pause to reproduce each concept in small components.

React and TypeScript: The Net Ninja search

Short lesson format; use it for concepts, then verify syntax against official docs.

Responsive CSS: Kevin Powell search

Focus on layout, responsive breakpoints, and maintainable CSS.

Git and GitHub: beginner course search

Learn working tree, staging, commits, branches, remotes, and push.

Deployment references

GitHub Actions: official quickstart

Learn workflows, triggers, jobs, and steps.

GitHub Pages: official quickstart

Understand the hosting platform used by this portfolio.

Pages with custom workflows

Matches this repo's deploy.yml workflow and artifact-based deployment.

Remember the pipeline

> Edit source -> run npm.cmd run build -> review git diff -> commit -> git push -> watch Actions -> refresh the live page after success.

The website is live at dag-gieh.github.io/portfolio. A green Pages workflow means GitHub published the new build.

Learning tip: understand one line of the existing site, make one change, run the build, and inspect the browser. Repeat. Keep claims and project outcomes accurate before publishing.